

Речь для защиты — HS:BG RL

Тайминг: ~7 минут 10 секунд на 13 слайдов. Лимит КР — 10 минут, рекомендация — 7.

Правила репетиции:

1. Прочитай вслух с секундомером, засеки каждый слайд отдельно
 2. Если слайд выходит дольше — сокращай. Не подгоняй сам себя в конце
 3. Ключевые слайды **не сокращай**: 6 (архитектура), 7 (главный результат)
 4. Запинки естественные, не звучи как диктор
 5. Обязательно тыкай пальцем в слайды 9–10 (attention) — даёт комиссии якорь
-

Слайд 1 — Титул (10 сек)

Здравствуйте. Меня зовут Пономарев Тимур, группа БПМИ243. Тема — применение обучения с подкреплением в играх жанра автобаттлеры.

Слайд 2 — Мотивация (45 сек)

Среда, на которой я работаю — Hearthstone Battlegrounds. Это автобаттлер от Blizzard, более 20 миллионов игроков в месяц. Игра состоит из двух чередующихся фаз: **recruit** — экономическая фаза, где игрок покупает юнитов, продаёт их, реролит магазин, апгрейдит таверну; и **combat** — автоматический бой двух досок.

Почему эта среда интересна именно как RL-задача? Это богатый testbed: здесь одновременно есть POMDP, потому что доску оппонента ты видишь только в бою, стохастика в самом бою, комбинаторный action space, и ещё поверх всего экономическое планирование. Структурно эта среда даже богаче, чем Pokemon из недавней работы Guo на RLC 2025 — там нет экономики и позиционирования. И важный момент — жанр живой, Blizzard добавляет новые карты каждый сезон, поэтому бенчмарк растёт вместе со средой.

Цель работы — разработать исследовательскую платформу и обучить агента.

Слайд 3 — Формализация POMDP (35 сек)

Формально это POMDP с $\gamma = 0.99$. Пространство наблюдений — вектор размерности 1009, структурированный по зонам: глобальный контекст, доска, рука, магазин, discovery и информация об оппоненте. Каждая сущность кодируется 37 признаками. Пространство действий — дискретное, 32 действия с динамическим маскированием нелегальных. Награда — многокомпонентная с терминальным плюсом-минус пять.

Слайд 4 — Почему нетривиально + gap (45 сек)

Почему задача нетривиальна: POMDP, стохастика, комбинаторика — порядка десяти в восьмой конфигураций доски при 18 юнитах на 7 слотов, sparse reward. Credit assignment классический.

Что касается существующих сред: Fireplace поддерживает классический Hearthstone, но не Battlegrounds. Unity ML-Agents медленные из-за графики. HSReplay — только реплеи без интерактивного управления. Ближайшие академические работы — Leiden 2022 и LUT 2024 — используют сильные

упрощения: без аур, без Discovery, без позиционных эффектов. Поэтому для исследования я написал собственный движок с полной боевой системой.

Слайд 5 — Симулятор (50 сек)

Движок на Python, на момент отчёта — около 3500 строк кода самого движка плюс 839 строк Gymnasium-обёртки. 11 модулей в пакете engine.

Ключевые подсистемы:

- **Entity System** с многоуровневым стеком баффов — базовый, перманентный, ходовой, боевой, аурный слой
- **Event System** с приоритизированной очередью событий и детерминированной сортировкой триггеров
- **Combat** — реализованы 10 ключевых механик, включая Deathrattle с цепными призывами, Reborn, Divine Shield, Cleave, Poisonous, Windfury, Magnetize, Avenge
- **Auras** с stateless пересчётом
- **Tavern** с триплетами, Discovery, заклинаниями

По охвату механик это наиболее полная реализация по сравнению с упомянутыми академическими работами.

Слайд 6 — Архитектуры MLP vs Transformer (80 сек) — КЛЮЧЕВОЙ

Дальше — архитектуры. Я сравнивал наивный MLP и Transformer-экстрактор.

Наивный MLP на плоском векторе R^{1009} плохо ложится на задачу по трём причинам: не различает семантические границы зон, не ловит взаимодействия между юнитами — расовые синергии или связку Baron с Deathrattle-юнитами, и тратит параметры на паддинг-слоты переменной доски.

Теперь как именно R^{1009} превращается в последовательность токенов для Transformer. Наблюдение структурировано по зонам: 7 слотов доски, 10 слотов руки, 7 слотов магазина и 3 слота discovery. Это **27 entity-слотов**, каждый размерности 37. Каждый слот проходит через DecomposedEncoder и становится одним entity-токеном размерности d_{model} , то есть 128. Дополнительно глобальный контекст из 7 признаков и информация об оппоненте из 3 конкатенируются и проходят через отдельный MLP, превращаясь в **обучаемый токен GLOBAL_CTX** — это 28-й токен. На выходе — **последовательность** 28×128 , которая идёт в стек из четырёх GTrXL-блоков и потом в PMA.

Теперь про компоненты, все из state-of-the-art работ:

- **DecomposedEncoder** из MARL-GPT кодирует разные группы атрибутов — continuous, binary, race — отдельными проекциями с суммированием
- **FiLM** из работы Perez 2018 — мультипликативная модуляция токенов глобальным контекстом
- Токен **GLOBAL_CTX** — аналог CLS из BERT и scalar features из AlphaStar, позволяет картам обращаться к макро-контексту внутри attention
- **GTrXL-шлюзы** из работы Parisotto 2020 со смещением минус три — ключ к стабильности: на старте шлюз почти закрыт, Transformer практически пропускает вход без изменений, агент учится простым вещам через линейные слои, а потом градиенты сами открывают шлюз

- **Multi-Seed PMA** из Set Transformer — четыре обучаемых seed-вектора для агрегации множества карт переменного размера

Компоненты все известны из литературы, но сборка именно под автобаттлеры — впервые.

Слайд 7 — Sample Efficiency (85 сек) — ГЛАВНЫЙ РЕЗУЛЬТАТ

Главный экспериментальный результат. Слева — реальные WandB-логи метрики board_power, среднее значение силы доски. Оранжевая линия — это Transformer, обучение остановлено на 300 тысячах шагов. Розовая — MLP, 1.5 миллиона шагов.

Видно: Transformer выходит на плато board_power около 15 примерно за 80 тысяч шагов, MLP достигает того же уровня за 400 тысяч. **То есть Transformer в пять раз быстрее по sample efficiency.**

Агент проходит три фазы обучения: сначала учится тратить золото, потом собирать синергии и триплеты, потом финальная шлифовка. Transformer проходит все три фазы в пять раз меньше шагов.

Про FPS — тут надо честно сказать. Transformer считает 250 шагов в секунду, MLP — 1200. Это per-step compute. Но wall-clock до плато у них уже примерно равен: у MLP 333 секунды, у Transformer 320. Почему так: MLP на плоской сети [512, 512] compute-bound на мелких матрицах, GPU у него простаивает, и добавление железа ему ничего не даст. Transformer же честно грузит GPU и масштабируется по batch size, по количеству устройств и по параметрам — тут работают стандартные scaling laws. **С ростом compute Transformer будет становиться только быстрее MLP, а не просто равен.**

Слайд 8 — PvP (35 сек)

Второй эксперимент — прямое столкновение Transformer против MLP. Три партии — все три ушли в таймаут более 500 ходов. HP колебалось в районе 20–30. Ни один не смог добить другого.

Моя интерпретация: обе архитектуры **решили текущую конфигурацию среды**. При 18 картах и тирах 1–3 пространство стратегий слишком узкое, чтобы архитектура дала преимущество. То есть движок работает корректно, нет эксплойтов, но нужна более сложная среда — и это мотивация для апдейтов после отчёта.

Слайд 9 — Attention по слоям (40 сек)

(тыкай пальцем/курсором в конкретные участки heatmap)

Третий результат — валидация через анализ attention-весов. Вот усреднённые карты внимания по слоям.

На **Layer 0** видно: внимание концентрируется **здесь** — внутри доски, B0 по B6 — и **вот здесь**, на токене GLOBAL_CTX. Модель смотрит сама на себя и сверяется с контекстом.

А на **Layer 3** — **вот здесь** появляются устойчивые cross-zone паттерны. Доска обращается к магазину. То есть модель на более глубоких слоях буквально задаёт вопрос «какая карта из shop подошла бы моей доске».

Слайд 10 — Attention heads (30 сек)

(тыкай в каждую голову)

Ещё интереснее — если разложить Layer 0 по отдельным головам. **Head 0** концентрируется внутри доски. **Head 1** смотрит на магазин. **Head 3** — глобальный контекст плюс Discovery. То есть головы **сами** специализировались на разных аспектах игрового состояния. Никто их к этому не принуждал — PPO-градиент сам нашёл разделение труда.

Слайд 11 — Reward + честность про ablation (40 сек)

Отдельно про reward shaping. Это признанная открытая задача RL, см. Ng 1999 про potential-based shaping и Andrychowicz HER. Эмпирически я пробовал: сначала 5-компонентный shaping из отчёта, потом MC Oracle PBRS через C++ симуляцию исходов боёв, потом откат к sparse reward плюс tier-based curriculum. Значимого стабильного выигрыша ни один не дал. Это и есть мой частичный ablation по reward.

Про покомпонентный ablation архитектуры. **В рамках вычислительного бюджета данного исследования полная покомпонентная абляция нецелесообразна** — фокус сделан на proof-of-concept архитектуры и валидации sample efficiency. Выбор компонент обоснован литературным обзором 60+ работ, документирован в репозитории.

Слайд 12 — Апдейты + Future Work (30 сек)

Коротко — что сделано после сдачи отчёта. Пул карт расширен с 18 до 175, тирь 1–7. Combat engine портирован на C++ через rubind11 — **270 тысяч боёв в секунду** на простых кейсах, 90 тысяч на сложных. Это в 40+ раз быстрее предыдущей C++-версии и примерно в **800 раз быстрее** чистого Python. Tier-based curriculum. Categorical Critic из DreamerV3. Кодовая база выросла с 3.5k до ~10.7k Python плюс 3.7k C++ плюс почти 10k тестов.

Дальше — авторегрессионный action space по схеме SAINT, RMCTS, пайплайн BC-PPO-Self-Play и opponent modeling.

Слайд 13 — Выводы (20 сек)

Резюмируя.

Первое — headless-среда HS:BG с наиболее полным охватом механик по сравнению с Leiden и LUT.

Второе — Transformer-экстрактор под автобаттлеры, новая сборка компонент из state-of-the-art работ 2018–2025 годов.

Третье — экспериментально подтверждено ускорение в 5 раз по sample efficiency, плюс attention-анализ показал самоорганизованную специализацию голов.

Спасибо за внимание.

Если надо сжать до 6 минут

- Слайд 4 → 30 сек (выкини Fireplace/Unity, оставь только Leiden/LUT)
- Слайд 8 (PvP) → 20 сек
- Слайд 11 (ablation) → 25 сек
- **Не трогай** слайды 6 и 7 — это ядро защиты

Критичные моменты

- Говори «я сделал», «**моя** среда», «я собрал» — чётко разграничивай своё и чужое (требование гайда Соколова)
- Для каждой чужой идеи называй фамилию + год: Perez 2018, Parisotto 2020, AlphaStar 2019, Set Transformer Lee 2019, MARL-GPT 2024
- На слайде 7 главная фраза — «**в 5 раз быстрее по sample efficiency**» — произнеси её громко и с паузой
- На слайде 11 не извиняйся за отсутствие полного ablation. Сформулируй как **ограничение вычислительных ресурсов** и **research trade-off**, обоснованный литобзором
- На слайдах 9–10 обязательно физически тыкай пальцем/курсором — без этого attention-картинки мёртвые